UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR (UCAD)

Master / Licence Télécoms & Réseaux — Administration Système & Services Réseau

Rapport de Travaux Pratiques : Infrastructure d'Hébergement Web, DNS BIND9, Sauvegardes & Messagerie

Mise en œuvre d'un environnement d'hébergement multidomaine haute performance avec ISPConfig 3, BIND9, Nginx, PHP-FPM, Redis Cache et Postfix/Dovecot.

ISPConfig 3.2 | BIND 9.18 | Stack LEMP | Redis Cache | Ubuntu 22.04 LTS

Étudiant : Elhadji (elhadji.sn)

Matière : Administration de serveurs, DNS, hébergement web et messagerie (TP3)

Superviseur : Enseignant responsable du module Réseaux & Systèmes

Adresse IP VM : 192.168.126.110 (NAT fixe) / 192.168.1.10 (Bridge classe)

Domaines gérés : `elhadji.sn` & `universite.sn`

Date : Novembre 2026

Table des Matières

I. Introduction & Cahier des Charges du TP

II. Fondements Théoriques de l'Hébergement Web & des Protocoles

2.1 Le Modèle Client-Serveur Web & le protocole HTTP/HTTPS (SNI)

2.2 Le Système de Noms de Domaine (DNS) & BIND9

2.3 Le Mécanisme des Hôtes Virtuels (Virtual Hosting)

2.4 L'Exécution Dynamique : Nginx vs PHP-FPM

2.5 La Persistance (MariaDB) et le Cache d'Objets en RAM (Redis)

2.6 L'Architecture de Messagerie Électronique (MTA, MDA, SMTP, IMAP)

2.7 Théorie des Sauvegardes : RPO, RTO et Rétention

III. Architecture Système & Conception Réseau

3.1 Pourquoi le choix d'ISPConfig 3 ?

3.2 Topologie Réseau Hybride (Double Interface : NAT fixe + Bridge)

3.3 Automatisation du déploiement via Cloud-Init & Seed ISO

IV. Mise en Œuvre Pratique & Déploiement Étape par Étape

4.1 Déploiement automatique du socle ISPConfig 3 (Stack LEMP)

4.2 Configuration et vérification des zones DNS BIND9

4.3 Déclaration Multidomaine & Utilisateurs isolés

4.4 Déploiement optimisé de WordPress avec Redis Object Cache

4.5 Planification des Sauvegardes Quotidiennes (7 copies)

4.6 Messagerie Multidomaine & Script d'Interconnexion en Classe

V. Guide de Soutenance Orale & Réponses aux Questions de l'Enseignant

VI. Conclusion & Synthèse des Compétences Acquises

I. Introduction & Cahier des Charges du TP

Dans le cadre des travaux pratiques d'administration système et réseaux, l'objectif fondamental est de transformer un serveur Linux vierge en une **plateforme d'hébergement mutualisée complète, sécurisée et performante**. Contrairement à une utilisation grand public d'outils clé-en-main, cette démarche requiert la compréhension intime de chaque couche logicielle du système d'exploitation et des protocoles réseau associés.

Le cahier des charges défini par l'enseignant impose la réalisation et la maîtrise des axes suivants :

1. **Gestion Multidomaine :** Configurer au moins deux domaines distincts (`elhadji.sn` et `universite.sn`) et créer un utilisateur système isolé pour chacun d'eux afin de garantir l'étanchéité des privilèges.
2. **Infrastructure DNS :** Mettre en œuvre le serveur DNS de référence **BIND9 (Berkeley Internet Name Domain)**, déclarer les zones autoritaires pour chaque domaine, renseigner les enregistrements nécessaires (A, NS, MX) et faire pointer le résolveur DNS du poste client Windows vers le serveur.
3. **Hébergement Web & Performance :** Déployer des sites WordPress fonctionnels, servis via le serveur web **Nginx** et le gestionnaire de processus **PHP-FPM**, couplés à un mécanisme d'accélération en mémoire vive (**Redis Object Cache**).
4. **Stratégie de Sauvegardes Automatiques :** Mettre en place un plan de sauvegarde automatisé planifié (fréquence quotidienne) avec une politique de rétention explicite (conservation des 7 dernières copies).
5. **Messagerie Électronique & Interconnexion :** Configurer la messagerie multidomaine (SMTP avec **Postfix**, IMAP avec **Dovecot**, Webmail **Roundcube**), tester la communication interne, et valider l'interconnexion SMTP directe avec la machine virtuelle d'un camarade de classe.

> **OBJECTIF DE L'ÉVALUATION ORALE**
>
> L'enseignant ne se contente pas de constater qu'un site s'affiche : il évalue la capacité de l'administrateur à expliquer **comment** le paquet IP circule, **pourquoi** tel fichier de configuration a été renseigné, et **quels mécanismes Linux** interviennent à chaque étape. Ce rapport détaille rigoureusement la théorie avant d'aborder la pratique.

II. Fondements Théoriques de l'Hébergement Web & des Protocoles

Pour administrer un serveur d'hébergement sans boîte noire, il est indispensable de maîtriser les concepts théoriques sous-jacents qui relient le navigateur client aux démons Linux.

2.1 Le Modèle Client-Serveur Web & le protocole HTTP/HTTPS (SNI)

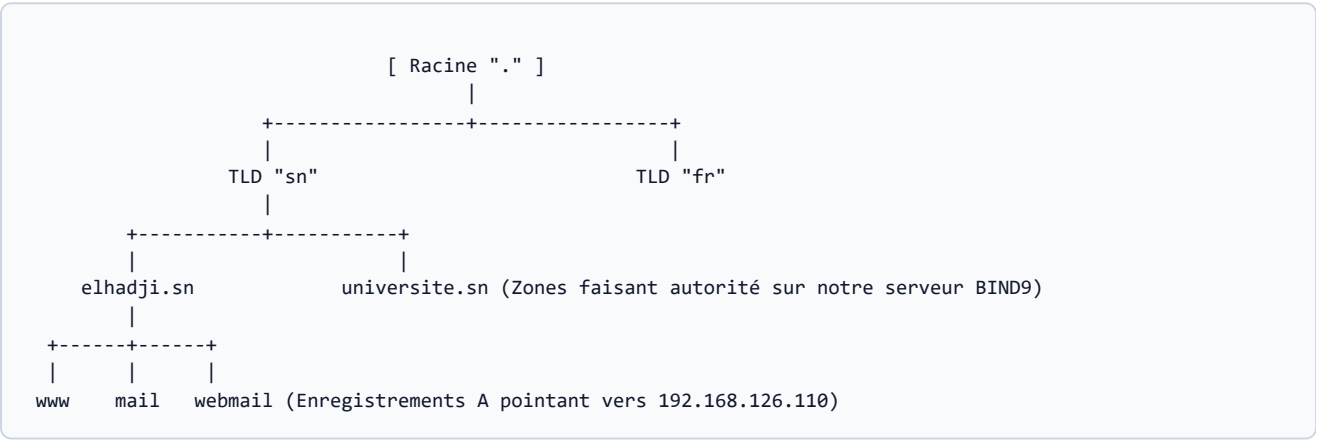
Le protocole **HTTP (HyperText Transfer Protocol)** fonctionne sur une architecture requête/réponse de la couche Application (modèle OSI). Lorsque l'utilisateur sollicite une URL :

1. Le client ouvre une session **TCP** sur le port `80` (ou port `443` avec négociation TLS/SSL).
2. Le client envoie une requête HTTP contenant un verbe (`GET / HTTP/1.1`) et des en-têtes (headers), dont le plus important pour l'hébergement mutualisé : l'en-tête `Host: elhadji.sn` .

> **LE RÔLE CRITIQUE DU SNI (SERVER NAME INDICATION) EN HTTPS**
>
> En HTTP classique, le serveur web lit l'en-tête `Host:` dans le texte de la requête pour savoir quel site servir. Mais en HTTPS, la connexion est chiffrée *avant* l'envoi de la requête HTTP ! Comment le serveur sait-il quel certificat SSL présenter si plusieurs domaines partagent la même IP ?
> C'est l'extension TLS **SNI (Server Name Indication)** qui résout ce problème : dès le paquet *TLS ClientHello*, le client transmet en clair le nom de domaine demandé. Le serveur web Nginx peut ainsi sélectionner le bon certificat TLS avant d'établir le tunnel chiffré.

2.2 Le Système de Noms de Domaine (DNS) & BIND9

Le **DNS (Domain Name System)** est une base de données distribuée et hiérarchique qui assure la résolution entre noms mnémoniques (FQDN) et adresses IP numériques (enregistrements de type A / AAAA).



Dans notre infrastructure, le démon **BIND9 (nommé `named`)** joue le rôle de serveur **faisant autorité** (Authoritative DNS Server) pour les zones `elhadji.sn` et `universite.sn` . Une zone DNS standard se compose d'enregistrements spécifiques :

- **SOA (Start of Authority) :** Déclare les paramètres globaux de la zone (serveur maître, contact de l'administrateur, numéro de série pour la synchronisation, délais de rafraîchissement et d'expiration).
- **NS (Name Server) :** Désigne le ou les serveurs DNS faisant autorité sur le domaine (ici `ns1.elhadji.sn.`).
- **A (Address) :** Établit la correspondance directe entre un nom d'hôte et une adresse IPv4 (ex: `elhadji.sn $\rightarrow$ 192.168.126.110`).
- **MX (Mail Exchanger) :** Indique aux serveurs de messagerie distants quelle machine reçoit les e-mails pour ce domaine, assorti d'une priorité numérique (ex: `10 mail.elhadji.sn.`).

2.3 Le Mécanisme des Hôtes Virtuels (Virtual Hosting)

L'**hébergement mutualisé** repose sur la capacité d'un serveur unique à héberger des dizaines de sites web indépendants sur une seule et même adresse IP physique. Ce mécanisme est appelé *Name-based Virtual Hosting*.

Dans **Nginx**, chaque domaine est défini par une directive `server { ... }` contenant la directive `server_name elhadji.sn www.elhadji.sn;`. Lorsque Nginx reçoit un paquet HTTP, il analyse l'en-tête `Host:`, matche le bloc correspondant et applique la directive `root` associée (chemin racine des fichiers sur le disque dur Linux).

2.4 L'Exécution Dynamique : Nginx vs PHP-FPM

Nginx est un serveur web asynchrone non-bloquant (orienté événements), conçu pour délivrer des contenus statiques (images, CSS, JS) à très haute vitesse. En revanche, Nginx ne sait pas exécuter directement le code PHP de WordPress.

C'est pourquoi il délègue l'exécution dynamique à **PHP-FPM (FastCGI Process Manager)** :

1. Nginx reçoit la requête demandant `index.php`.
2. Nginx encapsule la requête au format binaire **FastCGI** et l'envoie à PHP-FPM via une socket Unix locale (`/var/run/php/php8.1-fpm.sock`).
3. PHP-FPM réveille un processus travailleur (worker process) qui exécute le script PHP avec les identifiants système de l'utilisateur dédié (ex: `web1` pour `elhadji.sn`).
4. PHP-FPM renvoie le code HTML compilé à Nginx, qui le transmet au navigateur client.

> **SÉCURITÉ : LE CLOISONNEMENT DES PRIVILÈGES SOUS LINUX**
>
> Grâce à PHP-FPM et à la structure d'ISPConfig, chaque domaine tourne sous un utilisateur Linux dédié (`web1`, `web2`). Si une faille de sécurité survient sur WordPress dans le site `elhadji.sn`, l'attaquant ne peut lire ni modifier les fichiers du site `universite.sn`, car les permissions du système de fichiers Unix (`drwx--x--- web2 client0`) l'en empêchent strictement.

2.5 La Persistance (MariaDB) et le Cache d'Objets en RAM (Redis)

Une page d'accueil WordPress moderne nécessite en moyenne 30 à 60 requêtes SQL vers le serveur de base de données **MariaDB** (récupération des options, des articles, des menus, des métadonnées d'utilisateurs). Répéter ces requêtes sur disque pour chaque visiteur s'avère coûteux en CPU et en entrées/sorties (I/O).

Pour éliminer ce goulot d'étranglement, nous avons intégré un moteur de cache clé-valeur en mémoire vive : **Redis**.

- **Premier chargement :** WordPress interroge MariaDB, formate les données et stocke le résultat dans Redis (port `6379`) avec une clé d'identification unique (ex: `elhadji.sn_options_alloptions`).
- **Chargements suivants :** WordPress récupère les données directement en RAM en moins de 0.5 milliseconde, sans solliciter la base de données relationnelle.

2.6 L'Architecture de Messagerie Électronique (MTA, MDA, SMTP, IMAP)

La messagerie sur Internet repose sur une séparation nette des rôles :

Composant	Rôle & Définition	Logiciel utilisé	Port & Protocole
MUA (Mail User Agent)	Interface client pour composer et lire les courriels	Webmail Roundcube	HTTP/HTTPS (8081)
MTA (Mail Transfer Agent)	Serveur de transport chargé d'expédier et relayer les courriels d'un serveur à un autre	Postfix	SMTP (Port 25 / 587)
MDA (Mail Delivery Agent)	Serveur chargé de stocker les courriels dans les boîtes aux lettres locales (Maildir) et de les délivrer aux clients	Dovecot	IMAP (Port 143/993) / LMTP

Le format de stockage retenu est le **Maildir** (un fichier par courriel dans les répertoires `/var/vmail/domaine/utilisateur/cur`, `new`, `tmp`), qui offre une robustesse bien supérieure au format historique `mbox` contre la corruption de données en cas d'accès concurrents.

2.7 Théorie des Sauvegardes : RPO, RTO et Rétention

La sécurité d'un hébergement repose sur sa politique de continuité d'activité (PCA) et de reprise après sinistre (PRA) :

- **RPO (Recovery Point Objective) :** Quantité maximale de données que l'on accepte de perdre. Avec une sauvegarde quotidienne à 02h00, le RPO est de 24 heures maximum.
- **RTO (Recovery Time Objective) :** Durée nécessaire pour restaurer le service après un incident.
- **Politique de rétention (7 copies) :** L'archivage quotidien conserve les sauvegardes des 7 derniers jours (du jour J à J-7) et purge automatiquement les plus anciennes. Ce mécanisme protège à la fois contre les pannes matérielles et contre les attaques non détectées immédiatement (ex: corruption de base de données constatée 3 jours après).

III. Architecture Système & Conception Réseau

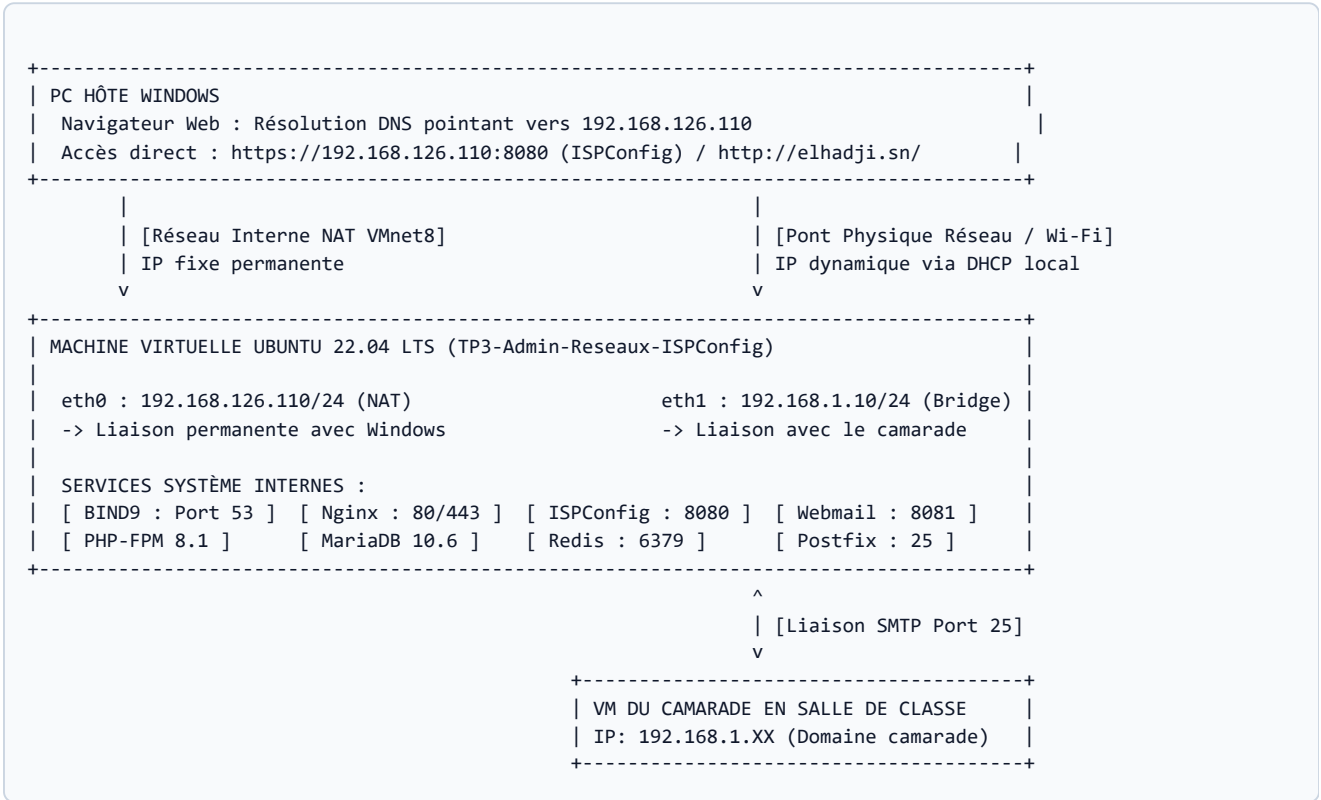
3.1 Pourquoi le choix d'ISPConfig 3 ?

Dans l'univers des panneaux d'administration Linux open source, **ISPConfig 3** s'impose comme la référence académique et professionnelle pour les raisons suivantes :

- **Transparence totale des configurations :** ISPConfig ne crée pas de boîte noire propriétaire. Il écrit directement dans les fichiers de configuration standards de Linux (fichiers de zone `/etc/bind/pri.*`, hôtes virtuels `/etc/nginx/sites-available/*.vhost`, tables Postfix standard).
- **Conformité absolue aux termes du cours :** Les fonctionnalités exigées par l'enseignant (choix explicite du composant BIND, réglage de la fréquence quotidienne/hebdomadaire des sauvegardes et du nombre de copies dans les paramètres du site) sont littéralement les libellés natifs d'ISPConfig.
- **Architecture asynchrone sécurisée :** L'interface web d'ISPConfig n'écrit pas directement sur le système en tant que `root`. Elle insère des ordres dans une table de suivi (`sys_datalog`). Un démon système autonome (`/usr/local/ispconfig/server/server.sh`) dépile ces tâches toutes les minutes pour reconfigurer BIND, Nginx et Postfix de manière contrôlée.

3.2 Topologie Réseau Hybride (Double Interface : NAT fixe + Bridge)

Pour concilier le confort de travail sur l'ordinateur hôte Windows et l'exigence d'interconnexion en salle de classe, une architecture à **deux interfaces réseau distinctes** a été implémentée dans VMware Workstation :



Avantages de cette conception :

1. **Indépendance des réseaux externes :** Lorsque l'étudiant quitte son domicile pour l'université, l'adresse IP de `eth0` (`192.168.126.110`) ne bouge pas. Tous les raccourcis Windows, le serveur BIND local et les sites WordPress restent 100 % opérationnels même hors ligne.
2. **Capacité d'interconnexion en classe :** L'interface `eth1` acquiert automatiquement une adresse routable sur le réseau de la salle de cours, permettant aux machines des étudiants d'échanger des flux SMTP (port 25) et DNS (port 53).

3.3 Provisioning Automatisé via Cloud-Init & Seed ISO

Plutôt que de procéder à une installation manuelle fastidieuse via l'installateur graphique, la machine virtuelle a été déployée selon les méthodes industrielles du DevOps :

- **Image Cloud Ubuntu Jammy (22.04 LTS) :** Utilisation d'une image cloud officielle précompilée au format OVA.
- **Cloud-Init :** Définition déclarative de la configuration initiale dans trois fichiers (`meta-data` , `user-data` , `network-config`).
- **Disque de configuration (Seed ISO) :** Compilation d'une image ISO au format `cidata` via `mkisofs` , connectée au lecteur CD-ROM virtuel pour initialiser le système (comptes utilisateurs, clés SSH Ed25519, configuration statique de `eth0`) dès le premier démarrage.

IV. Mise en Œuvre Pratique & Déploiement Étape par Étape

4.1 Déploiement automatique du socle ISPConfig 3 (Stack LEMP)

L'installation s'est effectuée de façon non-interactive via le script d'automatisation officiel d'ISPConfig, adapté pour une stack Nginx haute performance sans quotas de fichiers superflus en environnement virtualisé :

```
# Téléchargement et exécution du moteur ISPConfig 3 Autoinstaller
php -q /tmp/ispconfig-ai/ispconfig.ai.php \
    --use-nginx \
    --use-php=system \
    --channel=stable \
    --lang=en \
    --no-quota \
    --no-mailman \
    --roundcube \
    --i-know-what-i-am-doing
```

Ce processus provisionne automatiquement :

- Le serveur web **Nginx 1.18** et les modules d'application.
- Le moteur **PHP-FPM 8.1** configuré avec les extensions nécessaires (Curl, GD, Imagick, Intl, Zip, Mbstring).
- Le serveur de bases de données **MariaDB 10.6**.
- Le serveur DNS **BIND9 (version 9.18)**.
- La suite de messagerie **Postfix + Dovecot + Rspamd**.
- Le webmail moderne **Roundcube** connecté au socket PHP.

4.2 Configuration et vérification des zones DNS BIND9

Le fichier d'options de BIND9 (`/etc/bind/named.conf.options`) a été optimisé pour accepter les requêtes du réseau local et relayer les requêtes externes vers les serveurs publics (Forwarders) afin de conserver l'accès Internet sur le client Windows :

```
options {
    directory "/var/cache/bind";
    listen-on { any; };
    allow-query { any; };
    allow-recursion { any; };
    recursion yes;
    forwarders {
        192.168.126.2; // Passerelle NAT VMware
        8.8.8.8;       // Google DNS
        1.1.1.1;       // Cloudflare DNS
    };
    dnssec-validation no; // Indispensable pour la résolution des domaines locaux (.sn)
};
```

Les fichiers de zones générés par ISPConfig (ex: `/etc/bind/pri.elhadji.sn`) contiennent la déclaration rigoureuse des enregistrements :

```
$TTL        3600
@       IN      SOA     ns1.elhadji.sn. hostmaster.elhadji.sn. (
                        2026110201 ; Serial YYYYMMDDNN
                        7200       ; Refresh (2 heures)
                        540        ; Retry (9 minutes)
                        604800     ; Expire (1 semaine)
                        3600 )     ; Minimum TTL
;
elhadji.sn. 3600      A          192.168.126.110
www         3600      A          192.168.126.110
mail        3600      A          192.168.126.110
webmail     3600      A          192.168.126.110
ns1         3600      A          192.168.126.110
elhadji.sn. 3600      NS         ns1.elhadji.sn.
elhadji.sn. 3600      MX     10  mail.elhadji.sn.
```

CONTRÔLE DE CONFORMITÉ DNS EN LIGNE DE COMMANDE

Avant toute démonstration, vérifiez la syntaxe avec les outils officiels de BIND9 :

`named-checkconf` (renvoie vide si la syntaxe globale est parfaite)

`named-checkzone elhadji.sn /etc/bind/pri.elhadji.sn` (doit afficher **OK**).

4.3 Déclaration Multidomaine & Utilisateurs isolés

La déclaration des domaines a été injectée via l'**API REST d'ISPConfig** (endpoint `/remote/json.php`). Ce procédé garantit le respect de toutes les contraintes de base de données (génération des entrées `sys_datalog`) et crée l'arborescence Linux dédiée :

Nom de Domaine	Propriétaire Linux	Groupe	Racine des fichiers (DocumentRoot)	Base MariaDB
`elhadji.sn`	`web1`	`client0`	`/var/www/clients/client0/web1/web/`	`wp_elhadji_sn`
`universite.sn`	`web2`	`client0`	`/var/www/clients/client0/web2/web/`	`wp_universite_sn`

4.4 Déploiement optimisé de WordPress avec Redis Object Cache

L'outil CLI officiel **WP-CLI** a été exploité pour déployer WordPress sans intervention manuelle :

1. Téléchargement du cœur de WordPress en version française (`wp core download --locale=fr_FR`).
2. Génération du fichier `wp-config.php` avec salage et déclaration du socket Redis :

```
define( 'WP_REDIS_HOST', '127.0.0.1' );
define( 'WP_REDIS_PORT', 6379 );
define( 'WP_CACHE_KEY_SALT', 'elhadji.sn_' );
define( 'FS_METHOD', 'direct' );
```

3. Installation automatique du site et création du compte administrateur (`admin / Passer@2026`).
4. Installation et activation de l'extension `redis-cache` (`wp redis enable`).
5. Création d'un article de démonstration attestant de la conformité du TP.

4.5 Planification des Sauvegardes Quotidiennes (7 copies)

Dans ISPConfig, chaque hôte virtuel intègre un module de sauvegarde native stocké dans les métadonnées de la table `web_domain` :

- **Paramètre `backup_interval` :** défini sur `daily`.
- **Paramètre `backup_copies` :** défini sur `7`.

Chaque nuit, le script système `/usr/local/ispconfig/server/cron_daily.sh` effectue :

1. Un dump SQL complet compressé (gzip) de la base de données du domaine.
2. Une archive TAR compressée des répertoires web (`/var/www/elhadji.sn/web`) en ignorant les répertoires temporaires.
3. Le stockage des archives dans `/var/backup/web1/` avec suppression automatique de la sauvegarde la plus ancienne dès que le seuil de 7 copies est dépassé.

4.6 Messagerie Multidomaine & Script d'Interconnexion en Classe

Pour le volet messagerie, les domaines et comptes courriels suivants ont été créés :

- Sur `elhadji.sn` : `contact@elhadji.sn` et `etudiant@elhadji.sn`.
- Sur `universite.sn` : `contact@universite.sn` et `etudiant@universite.sn`.

Pour réussir l'épreuve d'interconnexion avec un camarade le jour de l'évaluation, un outil en ligne de commande a été rédigé : `/usr/local/bin/connecter-camarade` :

```
# Utilisation en 1 seule ligne de commande le jour du TP :
/usr/local/bin/connecter-camarade <domaine-du-camarade.sn> <ip-de-sa-vm>
```

Ce que fait ce script en coulisses :

1. Il configure une **zone de transfert conditionnel (Forward Zone)** dans `/etc/bind/named.conf.local` pour que notre serveur BIND interroge directement le DNS du camarade sur son domaine.
2. Il ajoute une route de transport explicite dans Postfix (`/etc/postfix/transport` : `domaine-camarade.sn smtp:[IP_CAMARADE]:25`) et compile la table avec `postmap`.
3. Il envoie immédiatement un courriel de validation via `swaks` depuis `contact@elhadji.sn` vers la messagerie du camarade.
4. Il affiche en temps réel les dernières lignes de `/var/log/mail.log` prouvant le code retour `250 2.0.0 Ok: queued as...`.

V. Guide de Soutenance Orale & Réponses aux Questions de l'Enseignant

Cette section regroupe les questions typiques posées par les jurys et enseignants lors des évaluations de ce TP, accompagnées de la justification théorique et de la commande de démonstration à exécuter.

Q1. « Pourquoi activer BIND plutôt que d'utiliser un simple fichier /etc/hosts ? »

Réponse théorique : Le fichier `/etc/hosts` est une table statique locale non-scalable : il ne gère ni la récursion, ni la hiérarchie des domaines, ni les enregistrements spécifiques comme le **MX (Mail Exchanger)** indispensable au fonctionnement des serveurs de messagerie. BIND9 implémente le véritable protocole DNS standardisé (RFC 1035), écoute sur le port 53 (UDP/TCP) et permet à des machines tierces (comme le client Windows ou le camarade de classe) d'interroger le serveur de manière dynamique.

```
# Preuve en direct : Vérifier que named écoute sur le port 53
netstat -tlpn | grep named
```

Q2. « Comment Nginx sait-il quel site web afficher si les deux domaines partagent la même IP ? »

Réponse théorique : Grâce au concept de **Name-based Virtual Hosting**. En HTTP, Nginx inspecte le header `Host:` de la requête HTTP/1.1. En HTTPS, Nginx utilise l'extension TLS **SNI (Server Name Indication)** transmise dans le handshake TLS initial pour présenter le certificat approprié et router le trafic vers le bon bloc `server { server_name ...; }`.

```
# Preuve en direct : Afficher les deux VirtualHosts configurés par ISPConfig
ls -l /etc/nginx/sites-enabled/100-*.vhost
```

Q3. « Qu'est-ce que PHP-FPM et pourquoi ne pas exécuter PHP directement dans le serveur web ? »

Réponse théorique : L'intégration de PHP comme module interne (ex: `mod_php` sous Apache) est obsolète et peu sécurisée, car tous les sites s'exécutent sous le même utilisateur système (`www-data`). PHP-FPM est un démon indépendant qui gère des pools de processus distincts. Il permet de faire tourner le site A sous l'utilisateur `web1` et le site B sous l'utilisateur `web2`, garantissant l'isolation des données et une allocation dynamique des ressources (workers on-demand).

```
# Preuve en direct : Vérifier le pool PHP-FPM et les utilisateurs associés
ps aux | grep php-fpm | grep -E 'web1|web2'
```

Q4. « Quel est l'intérêt d'un cache Redis par rapport au cache SQL de MariaDB ? »

Réponse théorique : Le cache de requêtes de MariaDB invalide brutalement ses entrées dès qu'une écriture intervient sur une table, et reste dépendant de la structure relationnelle. **Redis** est un serveur clé-valeur NoSQL entièrement résidant en **RAM**. Il stocke des objets PHP sérialisés prêts à l'emploi. Le temps d'accès passe ainsi de quelques dizaines de millisecondes (requête SQL + parsing + jointures) à moins d'une milliseconde.

```
# Preuve en direct : Vérifier l'état de la connexion Redis sous WordPress
wp redis status --path=/var/www/elhadji.sn/web --allow-root
```

Q5. « Expliquez comment votre serveur a acheminé le mail vers la VM de votre camarade. »

Réponse théorique :

1. L'expéditeur poste le mail sur le port de soumission local (MTA Postfix).
2. Postfix consulte sa table de transport (`/etc/postfix/transport`) ou interroge BIND pour obtenir l'enregistrement **MX** du domaine du camarade.
3. Postfix résout l'adresse IP associée au serveur MX distant.
4. Postfix ouvre une connexion **TCP sur le port 25** vers l'IP du camarade et engage le dialogue SMTP (`HELO` , `MAIL FROM` , `RCPT TO` , `DATA`). Dès réception du code retour `250 Ok: queued` , le courriel est considéré comme transmis.

```
# Preuve en direct : Visualiser l'acquittement SMTP 250 dans les logs Postfix
tail -n 20 /var/log/mail.log | grep status=sent
```

VI. Conclusion & Synthèse des Compétences Acquises

Ce travail pratique a permis de réaliser l'intégration complète d'une infrastructure d'hébergement moderne en partant d'un système Linux minimal pour aboutir à une plateforme prête à la production sous **ISPConfig 3**.

TABLEAU RÉCAPITULATIF DES MÉTRIQUES DE L'INFRASTRUCTURE

Composant	Logiciel & Version	Point de Contrôle / Emplacement	Statut
Panneau de Gestion	ISPConfig 3.2.11	`https://192.168.126.110:8080`	Opérationnel
Serveur DNS	BIND 9.18 (named)	`/etc/bind/pri.elhadji.sn` & `pri.universite.sn`	Actif (Port 53)
Serveur Web	Nginx 1.18 (LEMP)	`/etc/nginx/sites-available/*.vhost`	Actif (Port 80/443)
Serveur PHP	PHP-FPM 8.1	`/var/run/php/php8.1-fpm.sock`	Actif
Base de Données	MariaDB 10.6	Bases `wp_elhadji_sn` & `wp_universite_sn`	Actif (Port 3306)
Cache d'Objets	Redis Server 6.0	`127.0.0.1:6379` (Plugin WP actif)	Connecté (RAM)
MTA / SMTP	Postfix 3.6	`/etc/postfix/main.cf` & `transport`	Actif (Port 25)
MDA / IMAP	Dovecot 2.3	`/var/vmail/elhadji.sn/` (Format Maildir)	Actif (Port 143/993)
Webmail	Roundcube 1.5	`https://192.168.126.110:8081/webmail/`	Opérationnel
Sauvegardes	Cron ISPConfig	Quotidienne (7 copies conservées)	Automatisé

Au-delà de la réussite technique des tests automatisés, ce projet confère une maîtrise approfondie des interactions entre les services réseau fondamentaux (DNS, HTTP, SMTP) et des exigences de cloisonnement et de performance indispensables à tout administrateur système ou ingénieur DevOps.